Semana 3. Temporizadores del ESP32

1. Qué hace un temporizador de hardware

Un **timer** es un circuito que cuenta pulsos de reloj independientemente de loop(). Sirve para medir tiempo y generar un aviso cuando el contador alcanza una **alarma**. A diferencia de delay(), no necesita detener el programa; y a diferencia de consultar millis(), el evento lo detecta el hardware.

2. Frecuencia, prescaler y resolución

En los ejemplos de clase se utiliza un reloj de **80 MHz = 80 000 000 pulsos/s**. El **prescaler N** divide el reloj y determina la duración de cada tick del contador.

Magnitud	**Fórmula**
Frecuencia del contador	f = 80 000 000 / N
Resolución (tiempo por tick)	resolución = 1 / f
Tiempo de la alarma	t = alarma / f
Cuentas para la alarma	alarma = t × f

3. Ejercicios resueltos

Ejercicio 1: alarma cada 0,5 s. Se quiere trabajar con f = 1 MHz. Prescaler: N = 80 000 000 / 1 000 000 = **80**. Resolución: 1 / 1 000 000 = **1 µs**. Alarma: 0,5 × 1 000 000 = **500 000 cuentas**.

Ejercicio 2: frecuencia de 10 000 ticks/s y alarma en 2 500. Resolución: 1 / 10 000 = **100 µs**. Tiempo: 2 500 / 10 000 = **0,25 s**, por lo que hay **4 avisos/s**. Prescaler: 80 000 000 / 10 000 = **8 000**.

Ejercicio 3: 1 MHz y alarma de 2 000. t = 2 000 / 1 000 000 = **0,002 s = 2 ms**.

4. Alarma, autorrecarga y desbordamiento

La **alarma** compara el contador con un valor elegido. Con **autorrecarga**, cada vez que se alcanza ese valor se genera otro aviso y el conteo se reinicia. El **desbordamiento** ocurre al superar el máximo que permite el registro: $2^n - 1$. No es lo mismo que una alarma configurada.

El ESP32-WROOM-32 tiene **4 temporizadores** de propósito general en dos grupos (TIMG0 y TIMG1), con contadores de 64 bits. En el ATmega328P hay tres temporizadores, de 8 o 16 bits.

Ejemplo de límite: un contador de 8 bits llega a 255; si cada tick dura 1 µs, se llena en unos 0,26 ms. Uno de 16 bits llega a 65 535, aproximadamente 65,5 ms con la misma resolución.

5. De la alarma a la ISR

Cuando el timer alcanza la alarma puede generar una **interrupción**: la CPU atiende una función llamada **ISR** y luego continúa. La ISR debe ser corta, por ejemplo, marcar una bandera. Se utiliza **IRAM_ATTR** para ubicarla en memoria interna de instrucciones y **volatile** en variables que comparte con loop().

```
volatile bool aviso = false;
void IRAM_ATTR atenderTimer() {
  aviso = true;
}
void loop() {
  if (aviso) {
    aviso = false;
    // aqui se procesa el evento
  }
}
```

El fragmento solo muestra el **patrón bandera**: todavía habría que configurar el timer y asociar la ISR. La biblioteca de timers cambia entre versiones 2.x y 3.x del core de ESP32.

La función que se entrega a la API para ejecutarse cuando ocurre un evento se llama **callback**. Una interrupción de temporizador aparece al alcanzarse la alarma; una interrupción externa aparece cuando cambia la señal de un pin.

6. Más ejercicios breves

Datos	Operación	Respuesta
t = 10 ms; f = 1 MHz	0,010 × 1 000 000	10 000 cuentas
f = 40 000; alarma = 40 000	40 000 / 40 000	1 segundo
f = 40 000; reloj = 80 MHz	80 000 000 / 40 000	N = 2 000
f = 1 MHz; alarma = 500 000	500 000 / 1 000 000	0,5 s; 2 avisos/s

Ejercicio adicional: si f = 1 MHz y queremos avisos cada 750 ms, entonces 750 ms = 0,75 s y **alarma = 0,75 × 1 000 000 = 750 000**. Comprobación: 750 000 / 1 000 000 = 0,75 s.

Para resolverlos conviene escribir primero **f = reloj/N** y **t = alarma/f**, después sustituir valores y convertir correctamente segundos, milisegundos y microsegundos.